NanoBook: Depth Report

One stock's whole book, live: price levels (L2) and queues (L3), from its own replay · 26 October 2026 · C++23, GCC 14 / Clang 19, FTXUI 6.1.9, Linux · Built on the `ui` branch after the L1 viewer

1. Summary

The L1 viewer shows every stock's best bid and ask. This tool, `nanobook_depth`, shows **one stock's whole book**: every price level near the top, and the queue of orders waiting at each price, in the order they will fill. It rebuilds that book itself, from the same day's file, with the engine's own `FastBook`: a second copy, the way a trading firm keeps its own. The engine and the pipeline never know it exists.

Status	**Complete**: all 6 steps of depth design §6
The engine	**Untouched.** `git diff v1.0.0 -- include src` is empty; the book's existing read-only queries were enough
Live check	Phase 1's reference book gets every message too, compared after each one (§5.1)
The real day	AAPL, SPY, QQQ: **equal to a full replay of every stock at all 9 checkpoints** (the open, every hour, the end), the reference agreeing after each of their **8.2 million** messages (§5.3)
Speed	60× held on the real day (3 real seconds moved the clock 3 minutes); the replay and the drawing share one thread (§3.3)
Tests	**837** in Release builds, 818 with sanitizers (**18 new**), plus **8 new** layout tests in `nanobook_view_tests`; all 6 CI builds, format and clang-tidy clean
Planted bugs	**11** deliberate bugs (8 in the replay, 3 in the layout): **11 caught**

How to read this report:

- §2 explains the levels of a book and the queue in plain words, and §3 how the tool works.
- §4 shows what the real day looks like, §5 how correctness was proven.
- §6 lists the decisions and what changed, §7 how to run it, §8 what's next.

Numbers are from WSL2 on an Intel Core i7-14650HX. None is a latency claim: the tool measures nothing; it shows the book.

2. What it shows, in plain words

2.1 Three ways to look at one book

Picture a stall in a market. Buyers shout what they'll pay, sellers what they want:

Level	What you see	Like
L1	Only the best buyer and the best seller	The price tag in the window
L2	Every price, with the total shares wanted at each	The queue lengths at every counter
L3	Every single order, in its place in the queue	The names of the people in each queue

The L1 viewer shows L1 for every stock. This tool shows L2 and L3 for one.

2.2 The queue, and why "shares ahead" matters

At one price, orders fill in the order they arrived: **price-time priority**, a bakery queue. An order's **shares ahead** is everything in front of it: those shares must trade, or their orders be cancelled, before it fills. That one number is what a trading firm watches for its own orders, and what the queue table shows for every order.

2.3 Why its own replay

To copy a book you need every message, in order: miss one cancel and a gone order stays on screen for ever. The L1 viewer can skip updates (it only wants the latest); a book can't. Asking the pipeline for every message would either slow it down (it waits for the viewer) or lose messages (the copy goes wrong). So this tool doesn't ask: it reads the file itself, like a second firm reading the same feed.

---

3. How it works

3.1 The replay (`support/depth_replay.hpp`)

- **One message at a time.** `MessageCursor` hands the parser exactly one message, so the replay can stop between any two: at an ITCH time (`advance_to`) or after the stock's next message (`step`).
- **One stock.** Directory, trading-state and system messages go to the books for every stock (they name them); order messages only when their locate is the chosen stock's. Every stock's messages are parsed (368 million), which is simple and fast enough.
- **Two books.** The stock's book is a `FastBook`, sized by the same counting pass the pipeline uses, and a `ReferenceBook` gets the same messages (§5.1).
- **What the book doesn't keep,** the replay does: when each resting order arrived, who it was attributed to (F messages), the stock's last 64 messages (the tape), its last trade. An execution (E) carries no price; the price is the order's, from those notes.
- **Reading the book** uses `FastBook`'s read-only queries: `depth()` for the levels, `level_orders()` for a queue, `leaves()` for an order's shares. They allocate, which is why the engine never calls them while trading; here, in another process, ten times a second, that's fine.

3.2 The clock (`ReplayClock`)

`ITCH time = start + real time elapsed × speed`. Pause freezes it. Resuming or changing speed starts it again from the current ITCH time, so the screen never jumps; after steps taken while paused, it starts from where the steps got to.

3.3 One thread

Ten times a second the UI thread advances the replay to the clock's ITCH time, then draws. One thread touches the book, so nothing is locked and no read can tear. At 60× a tick is 6 s of the day, about 130,000 messages to parse, and the clock kept pace (§4.3): the tick fits in its 100 ms. Not timed more closely; it doesn't need to be.

---

4. What the real day looks like

4.1 A deep queue: AMD at 10:30

AMD (about $22) had many orders at each cent. Paused at 10:30:02, following the newest order at the best bid:

```
| NanoBook depth  AMD                                                    ITCH 10:30:02.704   1.0x   paused |
| bid 22.0800 x 1,400    ask 22.0900 x 2,900    spread 0.0100 (4.53 bps)    14,322 resting    state T        |
| following 72,130,821: 5th in line at bid 22.0800, 500 shares, 900 ahead of it                             |
|>        5        1,400       22.0800                | 22.0900              2,900        1                  |
|         8        2,100       22.0700                | 22.1000             22,504       30                  |
| QUEUE  bid 22.0800  (5 orders, 1,400 shares; front first)                                                 |
|      1      72,124,781         500             0  10:30:01.885                                           |
|      2      72,124,785         100           500  10:30:01.885                                           |
|      5      72,130,821         500           900  10:30:02.225                                           |
```

(Lines picked from the frame; the README shows it whole.) Twenty seconds later the followed order was **cancelled**, without filling: common for automated quotes, which move with the price.

4.2 One step, one message: AAPL

Paused at 09:41:30 and pressing `n` once applied one message, and the screen changed by exactly its effect: a new buy order at $162.01 became the best bid, the spread narrowed from 4 to 3 cents, resting orders went from 40,075 to 40,076, and the check count went up by one.

4.3 Driven by script

The tool was run in a detached terminal (tmux, 110 × 40) and every key pressed by script, saving the screen after each: 60× held (3 real seconds moved the clock 3 minutes), pause, step, a lower level's queue, follow (to "cancelled"), the ask side, `+` twice (240×) and `-` three times (30×), quit. All as designed.

---

5. How correctness was proven

5.1 The live check

Phase 4's `ReplayComparison`, reused: after every message of the stock, the two books' order count, best prices and the quantities there must agree (a quick check); every 1,024 messages, and for every stock at each system event, every level, queue and order's remaining shares (a full check). The first difference stops the replay right after the message that caused it, and the screen says so in red. A test plants one (a reference book too small to hold a third order) and checks the replay stops on exactly that order.

5.2 Tests

File	Tests	What
`tests/itch/test_depth_replay.cpp` (main suite)	18	Names and sizing; only the chosen stock reaches its book; levels and queues with shares ahead; executions, cancels, deletes and the tape; a replace losing its place; rejected messages; step and advance; the tape's cap; following an order to its fill and to its cancel; a broken message; a planted difference; the golden day's hand-checked book; a busy day equal to a full replay; the clock (speed, pause and resume, new speeds)
`tests/itch/test_depth_render.cpp` (`nanobook_view_tests`)	8	Frames drawn off-screen and read back: header, both ladders meeting in the middle, the queue's shares ahead, choosing a level and side, the tape newest first, the followed order, a difference shown, everything within 110 columns

5.3 The real day

`nanobook_depth FILE --check-day AAPL,SPY,QQQ` replays each stock next to a full replay of every stock, and compares the two copies of each stock's book in full at the open, every hour to 16:00 and the end:

Stock	Order messages	Quick checks	Equal at	Most resting
AAPL	1,656,597	1,656,597	9 of 9	42,767 orders (16:00)
SPY	3,055,956	3,055,956	9 of 9	2,920 orders (13:00)
QQQ	3,448,973	3,448,973	9 of 9	7,590 orders (16:00)

103 s in all (34 s of it the counting pass), 1.1 GB of memory. The first version compared only at the end, and passed, but proved nothing: Nasdaq clears every book at the close, so both copies were **empty**. The output said "0 orders resting", which is how it was caught (§6).

5.4 Planted bugs

Each was put in on its own, the tests run, and the code restored:

#	Bug	Caught by
1	The filter lets other stocks' orders through	5 tests (only the chosen stock, the tape, step, the cap, the planted difference)
2	"Shares ahead" counts the order itself	4 tests (queue, executions, replace, follow)
3	The tape drops replaces	A replace loses its place
4	An execution's price not taken from the order	Executions, cancels and the tape
5	A note kept after its order has left	Executions, cancels and the tape
6	Following stays on the old order after a replace	Follow an order to its fill
7	Resuming ignores steps taken while paused	The clock's pause and resume
8	Step stops after any message, not the stock's	Step applies one message of the stock
9	AHEAD column shows the order's own shares	Queue shows shares ahead (layout)
10	A replace's new id dropped from the tape line	Tape newest first (layout)
11	The two ladders swapped	Header and ladders (layout)

11 of 11 caught.

6. Decisions and what changed along the way

Decision	Chose	Instead of	Because
A	Its own replay of the file (option A)	The engine publishing depth (B), or every message sent out (C)	Zero risk to the engine and every published number; how firms really do it
D1	`FastBook` on screen, `ReferenceBook` checking it	One book	Shows the real engine, and the differential testing working live
D2	The counting pass to size the book	A guessed size	No guessing; about 30 s of start-up
D3	One thread	A replay thread	Nothing to lock; at 60× a tick fits in its 100 ms
D4	One stock per run	Switching inside the tool	A second terminal does it
D5	10 levels a side, `--levels N`		Fits a 40-row terminal with room for the queue and tape
D6	The `ui` branch, merged into `main` first	A new branch	The user's choice

What changed along the way, each recorded in the design note:

- `f` **follows the newest order** of the shown queue, not the front one: the newest has the longest way to go, which is what shows price-time priority working.
- **The real-day check became a mode of the tool** (no test reads the 11 GB file), and **compares through the day**, not only at the end, where both books are empty.
- **The trading state** comes from the replay's own copy of the H messages: the book builder keeps one without an accessor, and `include/` stays unchanged.
- **On screen:** "bid" and "ask" instead of the code's "Buy" and "Sell"; an order the replay never saw shows side "?" on the tape, not a guess.

7. How to run it

Build (needs the viewer's option, which downloads FTXUI):

```
cmake --preset gcc-release -DNANOBOOK_BUILD_VIEWER=ON
cmake --build build/gcc-release --target nanobook_depth
```

Watch one stock:

```
./build/gcc-release/nanobook_depth data/itch/01302019.NASDAQ_ITCH50/01302019.NASDAQ_ITCH50 AMD \
    --speed 1 --from 10:30:00
```

- About 30 s before the screen: the counting pass, then the stock replayed up to `--from`.
- `--speed` from 0.0625 to 1024 (default 1); `--from` defaults to 09:30:00; `--levels` to 10.
- Keys: up/down a level, Tab the side, Space pause, `n` step (one message of the stock), `f` follow the newest order in the shown queue (again to stop), `+`/`-` double or halve the speed, `q` quit.
- Next to the L1 viewer: give both the same `--speed` and `--from`; they show about the same moment (each paces from its own start).

Check it on the real day (no screen, about 2 minutes):

```
./build/gcc-release/nanobook_depth data/itch/01302019.NASDAQ_ITCH50/01302019.NASDAQ_ITCH50 \
    --check-day AAPL,SPY,QQQ
```

Tests: `ctest --preset gcc-release` runs them all, the layout tests included.

---

8. What's next

Nothing is planned. Possible later:

- **Charts:** a stock's queue lengths or spread over the day, recorded from the replay.
- **Watching the pipeline's own books** (options B or C of the design): engine-side work, so a measured experiment on bare metal, as the L1 viewer's step 4 was.